

Vigil Ponente

Cómo funciona el agente, explicado desde cero — esquema de trabajo, flujo de datos y glosario.

Proyecto Mitumi BackStage · Documento de referencia para personas que empiezan · Actualizado: 10/07/2026

■ Para quién es este documento

Está escrito para alguien que **no ha programado** este proyecto (o no ha programado nunca). No hace falta saber nada previo: cada término técnico se explica la primera vez que aparece, y al final hay un glosario. Si solo quieres una cosa: **los dos PDFs (uno para el ponente sin precios y otro para Mitumi con precios) se dibujan en el archivo pdf_report.py** (lo explicamos en la sección 7).

Contenido

1. Qué es Vigil Ponente, en una frase
2. Las 6 palabras que conviene entender primero
3. El recorrido de una búsqueda (diagrama)
4. El mismo recorrido, paso a paso
5. La regla de oro: al ponente nunca se le enseñan precios
6. Mapa de archivos: qué hace cada uno
7. Cómo se eligen las opciones y se generan los PDF
8. Qué datos maneja (los «moldes» de información)
9. Configuración: modo demo y modo real
10. Qué pasa cuando algo falla
11. Glosario completo

1 · Qué es Vigil Ponente, en una frase

Vigil Ponente es un **asistente que, a partir de un formulario con los datos del ponente y del evento, busca hotel, transporte (vuelos y trenes), taxi y coche de alquiler, y prepara dos informes en PDF** — sin reservar ni pagar nada, y sin leer ni escribir en ninguna base de datos.

La analogía más útil: imagina un **secretario de viajes** que, cuando le dices a quién y adónde va, reúne las opciones disponibles con su enlace de reserva y te prepara dos dosieres: uno para el ponente (sin precios) y otro para la organización de Mitumi (con precios). No compra los billetes ni firma nada por ti: reúne, ordena y redacta.

■ Lo que Vigil Ponente Sí hace

Entiende los datos del formulario, busca opciones de alojamiento y viaje con **enlace de reserva real**, deja que el organizador **marque varias opciones** (p. ej. 2 hoteles y 2 trenes) y con esa selección compila **dos PDF**: el del ponente (sin importes) y el de Mitumi (con importes y coste estimado).

■ Lo que Vigil Ponente NUNCA hace

Reservar, pagar o confirmar viajes; enviar correos por su cuenta; leer o modificar una base de datos; y, sobre todo, **mostrar precios al ponente**. El PDF del ponente se construye directamente sin ninguna cifra de importe — no es que se oculte, es que no se escribe.

2 · Las 6 palabras que conviene entender primero

Palabra	Qué significa aquí (en simple)
Formulario	La forma de entrada del agente. En vez de leer de una base de datos, recibe los datos rellenos en una web (o enviados por la plataforma). Cada búsqueda es independiente.
API / FastAPI	Una «puerta» por la que un programa habla con otro. Vigil Ponente tiene una API (el archivo <code>api.py</code> , hecha con FastAPI) con dos puertas principales: <code>/buscar</code> y <code>/generar_informe</code> .
LLM	«Modelo de lenguaje»: la IA que redacta texto. Aquí solo escribe el resumen del informe, y solo en el modo real (modelo Llama servido por Groq). No busca hoteles ni vuelos. En la demo, el resumen lo redacta una plantilla del código.
reportlab	La librería con la que se dibujan los PDF . Es quien pinta las tablas de hoteles, vuelos, trenes, etc. en el archivo <code>pdf_report.py</code> .
Modo demo	Un interruptor (<code>VIGIL_PONENTE_DEMO=1</code>) que hace que los resultados sean simulados y deterministas — sin proveedores reales ni IA. Es el modo de la presentación.
Propuesta	El «paquete» de resultados de una búsqueda (el objeto <code>PropuestaViaje</code>): dentro van las listas de hoteles, vuelos, trenes, taxis y coches encontrados, más la recomendación.

3 · El recorrido de una búsqueda (diagrama)

Cuando el organizador rellena el formulario, la petición atraviesa estos pasos. En cada caja se indica **qué archivo** se encarga de ese paso.



8

Se arma la propuesta con lo elegido y se dibujan los 2 PDF

`servicio.py → pdf_report.py`

9

Descarga: PDF del ponente (sin precios) + PDF de Mitumi (con precios)

`/informes/{id}/ponente.pdf · /informes/{id}/mitumi.pdf`

■ La clave del recorrido

Fijate en que la búsqueda (pasos 2–5) y la creación de los PDF (pasos 7–8) están **separadas**. Entre medias, el paso 6 es donde una persona decide qué entra en los informes. Es en el paso 8 donde se decide qué PDF lleva precios y cuál no.

4 · El mismo recorrido, paso a paso

Paso	Qué ocurre	Archivo	Nota para principiante
1	El organizador rellena el formulario y marca qué necesita.	<code>static/index.html</code>	En la demo, un desplegable precarga ponentes de ejemplo; en real, esos datos los inyecta la plataforma.
2	Se envía la búsqueda.	<code>api.py (/buscar)</code>	Viaja como una <code>SolicitudBusqueda</code> (los campos del formulario).
3	Se busca lo marcado.	<code>servicio.py → demo.py / sources.py</code>	Solo se busca cada servicio si su casilla está en «Sí».
4	Se recomienda y se resume.	<code>ranking.py · llm.py</code>	Elige la mejor opción y explica el porqué. La IA solo interviene en modo real.
5	Se devuelven las opciones.	<code>api.py</code>	Responde en JSON con la propuesta. Aún NO hay PDF.
6	El organizador marca varias opciones.	<code>static/index.html</code>	Cada tarjeta tiene una casilla; se pueden marcar varias por categoría.
7	Se envía la selección.	<code>api.py (/generar_informe)</code>	Viaja como una <code>SolicitudGenerarPDF</code> (listas de lo marcado).
8	Se dibujan los 2 PDF.	<code>servicio.py → pdf_report.py</code>	Aquí se decide qué lleva precios (ponente: no; Mitumi: sí).
9	Se descargan los informes.	<code>api.py (/informes/...)</code>	Los PDF quedan en disco con un id único y se sirven como descarga.

5 · La regla de oro: al ponente nunca se le enseñan precios

Es el punto de diseño más importante. De **una misma propuesta** se generan siempre **dos PDF** con la misma función (`pdf_report.generar_pdf`), cambiando un único interruptor `incluir_precios`:

<code>incluir_precios = False</code>	El informe del ponente . No lleva ninguna columna ni cifra de importe: ni precio por noche, ni total, ni coste estimado.
<code>incluir_precios = True</code>	El informe de Mitumi . Añade precios, el coste estimado (hotel + transporte) y la recomendación con importes.

■ Por qué se diseñó así

Para que el ponente **no elija siempre la opción más cara**. El precio no se «tapa» con un estilo visual: el PDF del ponente se construye desde cero sin datos de importe, así que es imposible que se filtre. Además, el «**porqué**» de la recomendación tiene dos versiones: `justificacion_ponente` (habla de comodidad, cercanía o rapidez, nunca de dinero) y `justificacion` (para el organizador, esta sí puede decir «es la más económica»).

6 · Mapa de archivos: qué hace cada uno

Agrupados por su papel en el recorrido. Todos viven dentro de la carpeta `vigil_ponente/`.

Entrada	<code>static/index.html</code> — la web (formulario + tarjetas + botón de PDF). <code>api.py</code> — la API: <code>/</code> , <code>/health</code> , <code>/buscar</code> , <code>/generar_informe</code> y las descargas <code>/informes/...</code>
Cerebro	<code>servicio.py</code> — orquesta la búsqueda y la compilación de los PDF. Es quien decide qué buscar y arma la propuesta.
Fuentes de datos	<code>demo.py</code> — resultados simulados y deterministas. <code>sources.py</code> — los proveedores reales (hoy sin implementar).
Recomendación	<code>ranking.py</code> — elige la mejor opción y redacta el porqué (con y sin precios). <code>fechas.py</code> — calcula llegada, salida y nº de noches.
Informes	<code>pdf_report.py</code> — aquí se dibujan los 2 PDF con reportlab.
Moldes de datos	<code>schemas.py</code> — las «fichas» de datos (<code>SolicitudBusqueda</code> , <code>PropuestaViaje</code> , <code>Hotel</code> , <code>OpcionTransporte</code> , <code>OpcionTaxi</code> , <code>OpcionCoche</code> , <code>SolicitudGenerarPDF</code>).
Configuración	<code>config.py</code> — carpeta de salida e interruptores. <code>serve_demo_ponente.py</code> — arranca la demo en el puerto 8001.
Ejemplos y pruebas	<code>examples/</code> — solicitudes de ejemplo. <code>tests/</code> — comprobaciones automáticas (27 en verde).

7 · Cómo se eligen las opciones y se generan los PDF

■ Respuesta directa

Los PDF se dibujan en `pdf_report.py`, pero **solo con las opciones que una persona ha marcado** en la web. La búsqueda y la generación de PDF son dos pasos distintos.

Conviene distinguir los dos momentos:

<code>/buscar</code>	Devuelve TODAS las opciones encontradas (p. ej. 3 hoteles, 3 vuelos, 3 trenes...) en JSON. Ya no genera ningún PDF . En la web, cada opción aparece como una tarjeta con una casilla.
----------------------	--

Selección (en la web)	El organizador marca las que quiere: puede elegir varias por categoría (2 hoteles, 2 trenes, 1 coche...). Un contador indica cuántas lleva marcadas.
/generar_informe	Recibe esa selección (el molde <code>SolicitudGenerarPDF</code> , con una lista por categoría). <code>servicio.generar_propuesta_pdf</code> arma una propuesta solo con lo elegido, calcula la recomendación y llama a <code>pdf_report.generar_ambos</code> .
Descarga	Los dos PDF se guardan en disco con un id único y se sirven en <code>/informes/{id}/ponente.pdf</code> y <code>/informes/{id}/mitumi.pdf</code> .

■ Detalle importante (es una demo)

En esta versión **no hay contraseña ni firma** en el enlace del PDF de Mitumi: es material de presentación, no datos reales. Tampoco hay base de datos ni histórico: si reinicias el servidor, los PDF antiguos siguen en disco pero el agente no guarda registro de búsquedas anteriores.

8 · Qué datos maneja (los «moldes» de información)

En vez de tablas de una base de datos, Vigil Ponente usa «moldes» (definidos en `schemas.py` con Pydantic) que describen la forma de cada dato. Estos son los principales:

SolicitudBusqueda	La entrada del formulario: <code>nombre_ponente</code> , <code>email</code> , <code>evento</code> , <code>ciudad_evento</code> , <code>fecha_inicio/fin</code> , <code>ciudad_origen</code> , <code>personas</code> , <code>preferencias</code> y 4 casillas <code>necesita_hotel</code> / <code>necesita_viaje</code> / <code>necesita_taxi</code> / <code>necesita_coche</code> .
PropuestaViaje	La salida de la búsqueda: listas de hoteles, vuelos, trenes, taxis y coches, más la recomendación (con y sin precios), el porqué y el <code>coste_estimado</code> .
Hotel	<code>nombre</code> , <code>estrellas</code> , <code>valoración</code> , <code>distancia al recinto</code> , <code>precio por noche y total</code> , y enlace de reserva.
OpcionTransporte	Sirve para vuelos y trenes: <code>proveedor</code> , <code>trayecto de ida y vuelta</code> (horarios, duración, escalas), <code>precio y enlace</code> .
OpcionTaxi	<code>proveedor</code> , <code>descripción del servicio</code> y <code>precio estimado</code> .
OpcionCoche	<code>compañía</code> , <code>categoría</code> , <code>oficina</code> , <code>precio por día</code> , <code>nº de días</code> y <code>precio total</code> .
SolicitudGenerarPDF	La selección curada : una lista por categoría (<code>hoteles_seleccionados</code> , <code>vuelos_seleccionados</code> , <code>trenes_seleccionados</code> , <code>taxis_seleccionados</code> , <code>coches_seleccionados</code>) con lo que el organizador marcó.

■ Cómo encajan

La `SolicitudBusqueda` entra por `/buscar` y produce una `PropuestaViaje`. El organizador elige un subconjunto, que viaja como `SolicitudGenerarPDF` a `/generar_informe` y termina en los dos PDF.

9 · Configuración: modo demo y modo real

VIGIL_PONENTE_DEMO=1	Activa el modo demo : resultados simulados y deterministas (la misma búsqueda da siempre lo mismo), sin proveedores ni IA. Es el modo de la presentación.
-----------------------------	--

Modo real	Habría que implementar <code>sources.py</code> (hoy lanza <code>NotImplementedError</code>) y definir <code>GROQ_API_KEY</code> para que el LLM redacte el resumen.
VIGIL_PONENTE_OUTPUT_DIR	Carpeta donde se guardan los PDF generados.
Puerto 8001	El de la demo del ponente. El otro agente del repo (Vigil de concursos) usa el 8000, para no chocar.
Dependencias	<code>fastapi</code> y <code>uvicorn</code> (la API), <code>pydantic</code> (los moldes), <code>reportlab</code> (los PDF) y <code>pypdf</code> (los tests).

10 · Qué pasa cuando algo falla

reportlab no está instalado	No genera PDF, deja un aviso en el log y la búsqueda sigue funcionando. Nunca rompe por esto.
Una fuente no devuelve nada	Esa sección sale vacía («Sin hoteles encontrados», etc.), pero el resto se muestra igual.
No marcas ninguna opción	La web avisa («Marca al menos una opción») y ni siquiera llama al servidor.
Falta la ciudad de origen con transporte	El botón «Buscar» se queda deshabilitado hasta que la indiques (taxi y coche no necesitan origen).
Modo real sin <code>sources.py</code>	Salta un <code>NotImplementedError</code> : por eso hoy la presentación va siempre en modo demo.

Glosario · completo

Agente

Un programa que recibe una petición, hace un trabajo acotado y devuelve un resultado. Vigil Ponente es el agente de logística de ponentes.

API

Conjunto de «puertas» por las que un programa pide cosas a otro. La de Vigil Ponente es `api.py`.

Endpoint

Una dirección concreta dentro de una API. Ej.: `/buscar` es el endpoint para pedir opciones.

FastAPI / uvicorn

FastAPI es la herramienta con la que está hecha la API; uvicorn es el servidor que la pone en marcha.

Formulario

La entrada del agente: los datos se rellenan en una web, no se leen de una base de datos.

LLM / Groq

La IA que redacta texto (modelo Llama servido por Groq). Aquí solo escribe el resumen, y solo en modo real.

reportlab

La librería que dibuja los PDF (tablas, textos, cabeceras).

Pydantic / schema

La forma de describir «cómo debe ser» un dato. Cada molde de `schemas.py` es un schema de Pydantic.

PropuestaViaje

El paquete con todas las opciones de una búsqueda.

Curación

El paso en que una persona elige, de entre todas las opciones, cuáles entran en los informes.

JSON

Un formato de texto para intercambiar datos entre programas. Las respuestas de la API viajan en JSON.

Modo demo

Interruptor que hace que los resultados sean simulados y repetibles, sin IA ni proveedores reales.

PDF ponente vs. Mitumi

Dos versiones del mismo informe: el del ponente sin precios y el de Mitumi con precios.

Documento de apoyo del agente Vigil Ponente — Mitumi BackStage. Refleja el estado del código a 10/07/2026. Para el detalle técnico exacto, la fuente de verdad son los archivos del propio proyecto (README.md y el código en vigil_ponente/).